https://colab.research.google.com/drive/1GKIFe1nZ8BMSABhd8iFochQMNlbFk4Mt?usp=sharing ex 6

```
!pip install -q faiss-cpu transformers torch pandas

import pandas as pd
import random

texts = [
"machine learning model",
"deep learning neural network",
"artificial intelligence system",
"data science analysis",
"natural language processing",
"python programming language",
"java backend development",
"web development html css",
"database management system",
"cloud computing service",
"image recognition algorithm",
"text classification model",
"speech recognition system",
"recommendation system engine",
"big data processing",
"software engineering practice",
"cyber security protection",
"mobile app development",
"computer vision detection",
"data visualization dashboard"
]

df = pd.DataFrame({"text": texts})

print("Dataset created successfully")
print(df.head())

TEXT_COLUMN = "text"
item_texts = df[TEXT_COLUMN].astype(str).tolist()

print("\nTotal items:", len(item_texts))
print("Sample item:", item_texts[0])

from transformers import AutoTokenizer, AutoModel
import torch
import numpy as np
import faiss

device = "cuda" if torch.cuda.is_available() else "cpu"

model_name = "sentence-transformers/all-MiniLM-L6-v2"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
encoder = AutoModel.from_pretrained(model_name).to(device)

encoder.eval()

def encode_texts(texts, batch_size=8):

    all_embeddings = []

    with torch.no_grad():

        for i in range(0, len(texts), batch_size):

            batch = texts[i:i+batch_size]

            tokens = tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = encoder(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)

            embeddings = embeddings.cpu().numpy().astype("float32")

            all_embeddings.append(embeddings)

    return np.vstack(all_embeddings)

item_embeddings = encode_texts(item_texts)

faiss.normalize_L2(item_embeddings)

print("\nEmbeddings shape:", item_embeddings.shape)

embedding_dim = item_embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dim)

index.add(item_embeddings)

print("FAISS index built with", index.ntotal, "items")

def get_similar_items(item_id, k=5):

    query_vector = item_embeddings[item_id:item_id+1]
```

```
    scores, indices = index.search(query_vector, k)

    return scores[0], indices[0]

query_id = 0

scores, idxs = get_similar_items(query_id, k=5)

print("\nQUERY ITEM:")
print(item_texts[query_id])

print("\nSIMILAR ITEMS:")

for score, idx in zip(scores, idxs):

    print(f"-> {item_texts[idx]} (score={score:.3f})")
```

```
Dataset created successfully
                            text
0          machine learning model
1    deep learning neural network
2  artificial intelligence system
3           data science analysis
4     natural language processing

Total items: 20
Sample item: machine learning model

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(

{"model_id":"14d892711971458fb69a3dbb49220a84","version_major":2,"vers
ion_minor":0}

Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.

{"model_id":"761ce15a9bb04a1bbf6a0782477e48fb","version_major":2,"vers
ion_minor":0}
```

{"model_id":"8ec3f3a926414f618e9607220e99dc59","version_major":2,"version_minor":0}

{"model_id":"c3d94caadd604b3c9fb3621aa2837947","version_major":2,"version_minor":0}

{"model_id":"67737ac9f49d4654ad0c65d6e0a16659","version_major":2,"version_minor":0}

{"model_id":"453cf7f91bc44482844d826d2fa5d671","version_major":2,"version_minor":0}

{"model_id":"c3de37b3eb34436fadd6846d546dc7e6","version_major":2,"version_minor":0}

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different
task/architecture; not ok if you expect identical arch.


Embeddings shape: (20, 384)
FAISS index built with 20 items

QUERY ITEM:
machine learning model

SIMILAR ITEMS:
-> machine learning model (score=1.000)
-> artificial intelligence system (score=0.680)
-> text classification model (score=0.604)
-> deep learning neural network (score=0.559)
-> natural language processing (score=0.475)
```